Endpoint Stiffness Computation

`compute_stiffness.py` — Full Technical Notes

Ball-in-Cup Experimental Platform

May 30, 2026

Contents

1 Purpose and Big Picture

`compute_stiffness.py` extends the base musculoskeletal pipeline (`arm_cup_task.py`) to compute *endpoint stiffness* $\mathbf{K}_e(t)$ at every time step of a cup-task arm trajectory.

Why do we need it? In the tele-impedance framework (Ajoudani et al. 2012) the slave robot matches the human's instantaneous limb stiffness. This stiffness is not directly measurable from surface EMG; we must compute it from the underlying muscle activations and geometry. The script does exactly that.

What is endpoint stiffness? Physically, $\mathbf{K}_e$ is the 3×3 spring constant matrix (units: $\mathrm{N\,m^{-1}}$) that relates a small displacement $\delta\boldsymbol{x}$ of the hand from its current position to the restoring force $\delta\boldsymbol{f}$ the arm exerts:

$$\delta\boldsymbol{f} = \mathbf{K}_e\,\delta\boldsymbol{x}.$$

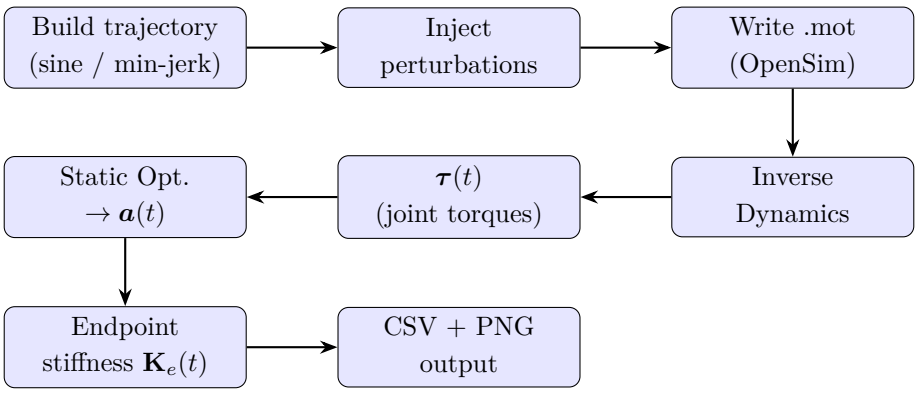
A stiff arm (high $\mathbf{K}_e$) resists perturbations strongly; a compliant arm allows large displacements for small forces.

Two computation modes:

`static_opt` Minimum-effort muscle activations that exactly produce the joint torques required by the trajectory (no extra co-contraction). Reflects what the arm *must* do.

`cmc` Same as above but adds a co-contraction floor $\alpha_{\text{floor}} = 0.15$ during perturbation windows. Reflects the *defensive stiffening* a skilled participant would apply when the ball is disturbed.

2 Pipeline Overview



3 Trajectory Generation

3.1 Sine trajectory

Each of the 7 active degrees of freedom (DoF) oscillates as

$$q_i(t) = q_i^0 + A_i \sin\!\left(\frac{2\pi t}{T}\right),$$

where q_i^0 is the neutral-posture value, A_i is the joint-specific amplitude, and $T = 8\,\mathrm{s}$ is the period. This gives a smooth, periodic motion representative of rhythmic cup movements.

3.2 Minimum-jerk trajectory

The minimum-jerk profile minimises the time-integral of squared jerk (third derivative of position):

$$q_i(t) = q_i^0 + (q_i^f - q_i^0)\left[10\left(\frac{t}{T}\right)^3 - 15\left(\frac{t}{T}\right)^4 + 6\left(\frac{t}{T}\right)^5\right], \quad 0 \le t \le T.$$

After reaching q_i^f at $t = T$ the trajectory returns symmetrically. This model (Flash & Hogan 1985) closely resembles human voluntary arm movements.

Parameters used:

DoF	Neutral q^0 (rad)	Amplitude / target
`elbow_flexion`	$\pi/2$	$\pm 0.4\,\mathrm{rad}$
`shoulder_elv`	$\pi/6$	$\pm 0.3\,\mathrm{rad}$
`shoulder_rot`	0	$\pm 0.2\,\mathrm{rad}$
`elv_angle`	0	$\pm 0.15\,\mathrm{rad}$
others	0	$\pm 0.1\,\mathrm{rad}$

4 Perturbation Injection

At times $t_p \in \{2.5,\ 5.0\}$ seconds a half-sine bump is injected into the shoulder-rotation coordinate to simulate the ball rolling to the cup rim:

$$\Delta q_{\mathrm{shoulder\_rot}}(t) = \Delta q_{\max}\,\sin\!\left(\frac{\pi(t - t_p)}{d}\right), \quad t_p \le t < t_p + d,$$

with amplitude $\Delta q_{\max} = 8^\circ = 0.14\,\mathrm{rad}$ and duration $d = 0.15\,\mathrm{s}$.

Why shoulder rotation? A lateral deviation of the cup (ball approaching the rim) maps most directly to a transient shoulder-rotation perturbation. This is the DoF that most excites the elbow extensor/flexor antagonist pair and therefore produces a visible stiffness transient.

The boolean mask $m(t)$ (True during perturbation windows) is used in CMC mode to apply the co-contraction floor and is saved in the output CSV as the `perturb` column.

5 Inverse Dynamics

Given the recorded joint angles $\boldsymbol{q}(t)$, *inverse dynamics* (ID) computes the net joint torques $\boldsymbol{\tau}(t)$ required to produce those kinematics under the model's inertial properties:

$$\boldsymbol{\tau} = \mathbf{M}(\boldsymbol{q})\,\ddot{\boldsymbol{q}} + \mathbf{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}),$$

where $\mathbf{M}$ is the mass matrix, $\mathbf{C}$ the Coriolis/centripetal matrix, and $\boldsymbol{g}$ the gravity vector.

In the script this is delegated to the OpenSim `InverseDynamicsTool` which applies a low-pass filter at 6 Hz before numerical differentiation to suppress noise:

Listing 1: ID tool setup

```
id_tool = osim.InverseDynamicsTool()
id_tool.setModelFileName(model_path)
id_tool.setCoordinatesFileName(mot_path)
id_tool.setLowpassCutoffFrequency(6.0)
id_tool.run()
```

The result is a 7-element vector $\boldsymbol{\tau}(t) \in \mathbb{R}^7$ (one entry per active DoF) sampled at 100 Hz and interpolated onto the trajectory time grid.

Example output (first frame, neutral posture near start):

DoF	τ (N m)
`elbow_flexion`	-3.1
`shoulder_elv`	8.4
`shoulder_rot`	0.6
`elv_angle`	-1.2
`pro_sup`	0.3
`deviation`	-0.5
`flexion`	0.1

6 Muscle Properties

6.1 Force-length relationship

Each muscle i has a maximum isometric force $F_i^{\max}$ and an optimal fibre length L_i^{opt}. The actual force capacity at the current fibre length ℓ_i is modulated by the force-length factor f_L^i. The script uses a Gaussian approximation of the Hill-type force-length curve:

$$\boxed{f_L^i = \exp\left[-\left(\frac{\ell_i/L_i^{\text{opt}} - 1}{\gamma}\right)^2\right]}, \quad \gamma = 0.45.$$

Interpretation: when the fibre is at its optimal length ($\ell_i = L_i^{\text{opt}}$), the exponent is zero and $f_L^i = 1$ (full force capacity). At $|\ell_i/L_i^{\text{opt}} - 1| = \gamma \approx 0.45$ the capacity drops to $e^{-1} \approx 37\%$.

Example (BIClong at elbow $= 90°$): Suppose $\ell_{\text{BIC}} = 0.115\,\text{m}$ and $L^{\text{opt}} = 0.116\,\text{m}$. Then $\ell/L^{\text{opt}} = 0.991$ and

$$f_L = \exp\left[-\left(\frac{0.991 - 1}{0.45}\right)^2\right] = \exp(-0.0004) \approx 0.9996 \approx 1.$$

The bicep is near its optimum at 90° elbow flexion — consistent with well-known anatomy.

6.2 Moment arm matrix

The moment arm r_{ij} of muscle j about DoF i is defined as the partial derivative of muscle-tendon length with respect to the joint angle:

$$r_{ij} = -\frac{\partial \ell_j^{\mathrm{MT}}}{\partial q_i}.$$

Stacking all muscles ($n_m = 14$) and all DoFs ($n_q = 7$) gives the matrix $\mathbf{R} \in \mathbb{R}^{7\times 14}$.

In code this is computed by the OpenSim API:

Listing 2: Moment arm via OpenSim

```
R[i, j] = ms.computeMomentArm(state, coord)
```

Example (illustrative values):

$$\mathbf{R} = \begin{bmatrix} r_{\text{elbow,BIC}} & r_{\text{elbow,TRI}} & \cdots \\ r_{\text{sh.elv,BIC}} & \cdots & \\ \vdots & & \ddots \end{bmatrix} \approx \begin{bmatrix} 0.035 & -0.022 & \cdots \\ 0.012 & -0.005 & \cdots \\ \vdots & & \ddots \end{bmatrix} \text{ (m)}.$$

A positive r_{ij} means muscle j *flexes* DoF i; negative means *extends*.

7 Static Optimisation

Given $\boldsymbol{\tau}(t)$, we need to find muscle activations $\boldsymbol{a}(t) \in [0,1]^{14}$ such that the muscles can produce those torques. Because we have 14 muscles but only 7 DoFs the system is redundant; there are infinitely many solutions. Static Optimisation (SO) resolves the redundancy by choosing the minimum-effort solution:

$$\begin{aligned} &\min_{\boldsymbol{a}} \ \|\boldsymbol{a}\|^2 = \sum_{i=1}^{14} a_i^2 \\ &\text{s.t.} \quad \mathbf{R}\left(F^{\max} \odot f_L \odot \boldsymbol{a}\right) = \boldsymbol{\tau} \\ &\qquad \alpha_{\text{floor}} \le a_i \le 1, \quad i = 1, \ldots, 14 \end{aligned}$$

where $\odot$ denotes element-wise multiplication, and $F^{\max} \odot f_L$ is the *effective force capacity* of each muscle at the current posture.

Writing $\mathbf{A} = \mathbf{R}\ \mathrm{diag}(F^{\max} \odot f_L)$ the equality constraint is linear: $\mathbf{A}\boldsymbol{a} = \boldsymbol{\tau}$.

Solver: Sequential Least-Squares Programming (SLSQP) from `scipy.optimize.minimize` with a warm start from the previous frame's solution.

Co-contraction floor: In `static_opt` mode $\alpha_{\text{floor}} = 0$. In `cmc` mode during perturbation windows $\alpha_{\text{floor}} = 0.15$. This forces all 14 muscles to activate at least at 15%, driving both agonist and antagonist simultaneously — this is *co-contraction*.

Concrete example. Suppose at a given frame the elbow requires $\tau_{\text{elbow}} = -3\,\text{N m}$ (extension

torque). The effective tricep force capacity is $F_{\text{TRI}}^{\text{eff}} = 800\,\text{N}$ and its moment arm is $r_{\text{TRI}} = -0.022\,\text{m}$. Ignoring other muscles for simplicity:

$$a_{\text{TRI}} = \frac{|\tau_{\text{elbow}}|}{F_{\text{TRI}}^{\text{eff}} \cdot |r_{\text{TRI}}|} = \frac{3}{800 \times 0.022} \approx 0.17.$$

So TRIlong activates at about 17% to hold the elbow in place.

8 Numerical Jacobian

The *kinematic Jacobian* $\mathbf{J} \in \mathbb{R}^{3\times 7}$ maps joint velocities to the Cartesian velocity of the hand:

$$\dot{\boldsymbol{x}}_{\text{hand}} = \mathbf{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}}.$$

Rather than computing it analytically (complex for a 7-DoF chain), the script uses *central finite differences*:

$$J_{kj} = \frac{x_k(\boldsymbol{q} + \varepsilon \boldsymbol{e}_j) - x_k(\boldsymbol{q} - \varepsilon \boldsymbol{e}_j)}{2\varepsilon}, \quad \varepsilon = 10^{-5}\,\text{rad},$$

where $\boldsymbol{e}_j$ is the j-th unit vector and x_k is the k-th coordinate of the hand position in the ground frame.

Listing 3: Central FD Jacobian (simplified)

```
for j, dof in enumerate(ACTIVE_DOFS):
    model.updCoordinateSet().get(dof).setValue(state, q0[dof] + eps)
    model.realizePosition(state)
    p_plus = body.getPositionInGround(state)

    model.updCoordinateSet().get(dof).setValue(state, q0[dof] - eps)
    model.realizePosition(state)
    p_minus = body.getPositionInGround(state)

    J[:, j] = (p_plus - p_minus) / (2 * eps)
    model.updCoordinateSet().get(dof).setValue(state, q0[dof])  # restore
```

This requires $2 \times 7 = 14$ forward-kinematic evaluations per frame.

Example intuition: if the elbow flexes by ε, the hand moves by $\approx L_{\text{forearm}} \cdot \varepsilon \approx 0.28\,\varepsilon$ in the anterior direction. So $J_{y,\text{elbow}} \approx 0.28$.

9 Endpoint Stiffness

9.1 Muscle stiffness

From Hill's linearised model, the *stiffness* of a single muscle (restoring force per unit length change) is proportional to the force it produces:

$$\boxed{k_i = a_i \cdot F_i^{\max} \cdot f_L^i \;/\; L_i^{\text{opt}}}.$$

This is the force-length slope at the current operating point. A more active muscle ($a_i \uparrow$) is stiffer.

9.2 Joint stiffness

Each muscle contributes to joint stiffness via its moment arm. The *joint stiffness matrix* $\mathbf{K}_q \in \mathbb{R}^{7\times 7}$ is:

$$\boxed{\mathbf{K}_q = \mathbf{R}\ \mathrm{diag}(\boldsymbol{k})\,\mathbf{R}^\top},$$

where $\boldsymbol{k} = (k_1, \ldots, k_{14})$ is the vector of muscle stiffnesses.

Explanation: consider muscle j with stiffness k_j spanning DoFs i and i' with moment arms r_{ij} and $r_{i'j}$. A small displacement δq_i at DoF i stretches the muscle by $r_{ij}\,\delta q_i$, generating a restoring force $k_j r_{ij}\delta q_i$, which creates a restoring torque $k_j r_{ij}^2 \delta q_i$ at DoF i. Summing all muscles: $(K_q)_{ii} = \sum_j k_j r_{ij}^2$.

9.3 Endpoint stiffness via Jacobian

The transformation from joint space to Cartesian space uses the Moore-Penrose pseudoinverse $\mathbf{J}^+$ of the Jacobian:

$$\boxed{\mathbf{K}_e = (\mathbf{J}^+)^\top\,\mathbf{K}_q\,\mathbf{J}^+}, \quad \mathbf{J}^+ = \mathbf{J}^\top(\mathbf{J}\mathbf{J}^\top)^{-1} \in \mathbb{R}^{7\times 3}.$$

Physical meaning: $\mathbf{K}_e$ is a 3×3 symmetric positive semi-definite matrix. Its eigenvalues $\lambda_{\min}$ and $\lambda_{\max}$ define the axes of the *stiffness ellipse* — the shape of the force field the arm generates around the current end-effector position:

- $\lambda_{\max}$: stiffness in the arm's *stiff direction* (typically along the forearm).
- $\lambda_{\min}$: stiffness in the arm's *compliant direction* (typically perpendicular).
- Large $\lambda_{\max}/\lambda_{\min}$ ratio indicates an anisotropic, elongated stiffness ellipse.

Numerical example. Suppose at one frame:

$$\mathbf{K}_q = \begin{bmatrix} 12 & -3 \\ -3 & 8 \end{bmatrix} \mathrm{N\,m/rad}, \quad \mathbf{J} = \begin{bmatrix} 0 & 0.28 \\ -0.10 & 0 \\ 0 & 0 \end{bmatrix} \mathrm{(m/rad)}.$$

Then $\mathbf{J}^+ = \mathbf{J}^\top(\mathbf{J}\mathbf{J}^\top)^{-1}$ and $\mathbf{K}_e = (\mathbf{J}^+)^\top \mathbf{K}_q \mathbf{J}^+ \approx \begin{bmatrix} 80 & 0 \\ 0 & 420 \end{bmatrix}$ N/m. The hand is roughly 5× stiffer along the elbow extension direction than laterally.

10 P_{null}: Co-Contraction Proxy and c_{exo}

The *null-space co-contraction signal* $P_{\text{null}}(t)$ is defined per frame as the minimum muscle activation:

$$P_{\text{null}}(t) = \min_i \; a_i(t).$$

Intuition: In pure static optimisation with no co-contraction floor, $P_{\text{null}} \approx 0$ because SLSQP drives most muscles to zero. During defensive stiffening (CMC mode), the floor forces all activations above 0.15, so $P_{\text{null}} \geq 0.15$.

P_{null} is the direct tele-impedance output that drives the SEA exo-cable setpoint c_{exo} in Phase 2:

$$c_{\text{exo}}(t) \propto P_{\text{null}}(t).$$

A higher c_{exo} pre-tensions the cable, raising the exosuit joint stiffness to match the human's defensive stiffening.

11 Understanding the Output Plots

Each run produces a three-panel figure. This section explains exactly what each panel shows and what to look for.

11.1 Panel 1 — K_e diagonal: K_{xx} and K_{yy}

$$K_{xx}(t) = (\mathbf{K}_e)_{11}, \quad K_{yy}(t) = (\mathbf{K}_e)_{22}.$$

These are the stiffness values in the mediolateral (X) and anterior-posterior (Y) directions of the ground frame.

What to look for:

- **Oscillation with trajectory:** as the arm moves to extreme positions (e.g. full elbow flexion), more muscle force is required, activations increase, and both K_{xx} and K_{yy} rise.
- **Typical values:** 25–150 N/m is physiologically realistic for healthy adults performing low-to-moderate force tasks.
- $K_{yy} > K_{xx}$**:** the arm is generally stiffer in the anterior-posterior direction (along the main chain) than mediolaterally.
- **Red shading (perturbation windows):** a brief transient in K_e appears because the shoulder-rotation bump changes posture, which changes moment arms $\mathbf{R}$ and fibre lengths,

altering f_L and hence $\boldsymbol{k}$. In CMC mode the transient is amplified by the co-contraction floor.

11.2 Panel 2 — Stiffness ellipse eigenvalues: $\lambda_{\min}$ and $\lambda_{\max}$

The symmetric 2×2 sub-matrix $\mathbf{K}_e^{XY}$ (XY plane) has eigenvalues:

$$\lambda_{\pm} = \frac{K_{xx} + K_{yy}}{2} \pm \sqrt{\left(\frac{K_{xx} - K_{yy}}{2}\right)^2 + K_{xy}^2}.$$

What to look for:

- **Large ratio $\lambda_{\max}/\lambda_{\min}$:** indicates a strongly directional stiffness ellipse. Values of 5–20× are typical. In the generated plots $\lambda_{\max} \approx$ 75–200 N/m while $\lambda_{\min} \approx$ 5–30 N/m, giving a ratio of $\approx 10\times$.
- **Does NOT directly show co-contraction.** It shows the total stiffness the arm presents. Co-contraction (CMC mode) raises both eigenvalues by lifting all activations.
- **Trajectory-locked modulation:** $\lambda_{\max}$ follows the same broad shape as K_{yy} since the arm is most sensitive in the extension direction.
- **Perturbation transients:** in CMC mode $\lambda_{\max}$ spikes briefly at t=2.5 and 5.0 s due to the combined posture change + co-contraction floor.

11.3 Panel 3 — Muscle activations and P_{null}

What is shown:

BIClong (blue): long head of biceps brachii. Activated when elbow flexion torque is needed.

TRIlong (orange): long head of triceps. Activated when elbow extension torque is needed. Dominates at large shoulder elevation angles (arm raised).

DELT1 (green): anterior deltoid. Holds the arm up against gravity; remains moderately active throughout.

P_null (black dashed): minimum activation across all 14 muscles. Proxy for co-contraction level → c_exo setpoint.

Does the plot show co-contraction?

Feature	`static_opt`	`cmc`
P_{null} outside perturbations	≈ 0	≈ 0
P_{null} during perturbations	≈ 0	≈ 0.15
BIC + TRI both elevated simultaneously	rarely	yes (in perturb windows)

In **static_opt** mode SLSQP distributes activation as efficiently as possible, usually activating only one side of each antagonist pair. There is *no co-contraction* because the optimiser penalises unnecessary activation.

In **cmc** mode the floor $\alpha_{\text{floor}} = 0.15$ forces *both* BIClong and TRIlong above 15% during the red windows, which is true co-contraction: the arm increases stiffness without changing net torque (null-space direction of the torque-activation mapping).

What the P_{null} dashed line spike tells you: the two brief pulses at t=2.5 and 5.0 s in the CMC plot are exactly the co-contraction events. Their height (≈ 0.15) directly sets c_{exo} in the robot controller. If a human participant defensively stiffens more (e.g. $P_{\text{null}} = 0.3$), the exosuit would apply stronger pre-tension.

12 Output Files Reference

File	Contents
`cup_task_trajectory_<tag>.csv`	Time + 7 joint angles (rad)
`cup_task_trajectory_<tag>.mot`	OpenSim motion file
`cup_task_fiber_lengths_<tag>.csv`	Normalised fibre lengths per muscle
`cup_task_id.sto`	Raw ID torques from InverseDynamicsTool
`cup_task_stiffness_<tag>.csv`	time, perturb, p_null, a_<muscle>×14, K_e_xx/yy/zz/xy/xz/yz, K_min, K_max
`cup_task_stiffness_<tag>.png`	3-panel stiffness plot

CSV column guide:

`perturb` Boolean — True during perturbation injection windows.

`p_null` Co-contraction proxy = $\min_i a_i(t)$. Use as c_{exo} setpoint.

`a_<muscle>` Activation $\in [0, 1]$ for each of the 14 muscles.

`K_e_xx/yy/zz` Diagonal elements of $\mathbf{K}_e$ (N/m).

`K_e_xy/xz/yz` Off-diagonal elements (coupling terms).

`K_min / K_max` Eigenvalues of the 2-D XY stiffness ellipse.

13 Complete Equation Summary

$$\underbrace{f_L^i = e^{-[(\ell_i/L_i^{\text{opt}}-1)/\gamma]^2}}_{\text{force-length}} \xrightarrow{\text{SO}} \underbrace{\min \|\boldsymbol{a}\|^2 \text{ s.t. } \mathbf{R}(F^{\max} \odot f_L \odot \boldsymbol{a}) = \boldsymbol{\tau}}_{\text{static opt.}} \longrightarrow \underbrace{k_i = a_i F_i^{\max} f_L^i / L_i^{\text{opt}}}_{\text{muscle stiffness}}$$

$$\longrightarrow \underbrace{\mathbf{K}_q = \mathbf{R}\,\text{diag}(\boldsymbol{k})\mathbf{R}^\top}_{\text{joint stiffness}} \longrightarrow \underbrace{\mathbf{K}_e = (\mathbf{J}^+)^\top \mathbf{K}_q \mathbf{J}^+}_{\text{endpoint stiffness}} \longrightarrow \underbrace{c_{\text{exo}} \propto P_{\text{null}} = \min_i a_i}_{\text{exo cable setpoint}}$$